

PFU-111 PROGRAMACIÓN FUNDAMENTAL

AUDITORÍA DE SEGURIDAD DE CÓDIGO EXISTENTE

Grupo 2

Actividad: Evaluación de código existente y prevención de vulnerabilidades comunes

Integrantes:

RONALD CRUZ CHINO

Denilson Davis Machaca Flores

Daniel Wilfredo Pomarez Huaranca

Fecha: 8 de octubre de 2026

1. Objetivo

Analizar un programa de registro de usuarios para identificar riesgos relacionados con la validación de entradas, el manejo de errores y el control de datos, y posteriormente implementar medidas preventivas que permitan que el programa procese únicamente valores válidos.

2. Descripción del código analizado

El programa inicial solicita por teclado el nombre, la edad, una nota y una opción. Después determina si la persona es mayor o menor de edad, muestra la nota y ejecuta una acción según la opción elegida. El problema principal es que los datos introducidos por el usuario se utilizan directamente, sin comprobar suficientemente su tipo, rango o formato.

La guía de la actividad indica que la auditoría debe considerar entradas no validadas, valores fuera de rango, formatos incorrectos, ausencia de manejo de errores y opciones no controladas. La actividad también exige comparar la versión original con una versión mejorada y demostrar las pruebas realizadas.

3. Investigación de riesgos y vulnerabilidades

La validación de entradas consiste en comprobar que los datos cumplen las reglas que necesita el programa antes de utilizarlos. OWASP recomienda validar tipo, formato, rango y valores permitidos, y rechazar los datos que no cumplen las reglas definidas. [\[cite turn0search0\]](#)

En Python, una entrada que no puede convertirse al tipo solicitado puede producir una excepción. La documentación oficial muestra el uso de try/except para controlar errores como ValueError y volver a solicitar un dato válido. [\[cite turn0search1\]](#)

4. Tabla de vulnerabilidades

Parte del código	Problema	Posible consecuencia	Nivel
Edad	No se controla el rango.	Una edad negativa o excesiva puede ser aceptada.	Media
Nota	No se comprueba que esté entre 0 y 100.	Se pueden registrar notas inválidas.	Media
Conversión de edad/nota	int() y float() se ejecutan sin try/except.	Una entrada no numérica puede detener el programa.	Alta
Opción	No se limita el dato antes de procesarlo.	Puede producir una selección no prevista o un error de conversión.	Media
Nombre	No se comprueba vacío ni longitud.	Se pueden registrar datos incompletos o excesivamente largos.	Baja

5. Clasificación del riesgo

Problema	Probabilidad	Impacto	Nivel	Justificación
Entrada no numérica en edad/nota	Alta	Alto	Alta	Puede detener la ejecución por una excepción no controlada.

Valores fuera de rango	Media	Medio	Medio	El programa puede continuar con información que no tiene sentido.
Opción no válida	Media	Medio	Medio	El usuario puede seleccionar un valor que no corresponde a las opciones definidas.
Nombre vacío o excesivo	Baja	Bajo	Bajo	Afecta la calidad del dato y puede dificultar su posterior procesamiento.

6. Mejoras preventivas

Problema	Solución propuesta	Estrategia preventiva	¿Qué mejora?
Edad fuera de rango	Aceptar solo edades entre 0 y 120.	Validación de rango	Evita datos imposibles o poco razonables.
Nota fuera de rango	Aceptar solo notas entre 0 y 100.	Validación de rango	Impide registrar una calificación inválida.
Entrada no numérica	Usar try/except y volver a solicitar el dato.	Manejo de excepciones	Evita que el programa termine inesperadamente.
Opción inválida	Aceptar únicamente 1, 2 o 3.	Lista/rango permitido	Mantiene el flujo dentro de las opciones definidas.
Nombre vacío/largo	Exigir entre 1 y 50 caracteres.	Restricción de longitud	Mejora la calidad y control del dato.

7. Implementación

La versión mejorada fue desarrollada en Python. Se separaron las validaciones en funciones para hacer el programa más claro y reutilizable. Los datos solo pasan al procesamiento principal después de superar las validaciones.

Ejemplo de manejo de errores implementado:

```
try:
    valor = int(input(mensaje))
    if minimo <= valor <= maximo:
        return valor
    print("Error: valor fuera de rango.")
except ValueError:
    print("Error: debe ingresar un número entero válido.")
```

8. Pruebas de seguridad

Prueba	Entrada utilizada	Resultado esperado	¿Superó?
Entrada válida	Nombre: Ronald / Edad: 19 / Nota: 85 / Opción:	Datos aceptados y proceso normal.	Sí

	1		
Edad negativa	Edad: -5	Solicitar nuevamente la edad.	Sí
Edad excesiva	Edad: 200	Solicitar nuevamente la edad.	Sí
Nota fuera de rango	Nota: 120	Solicitar nuevamente la nota.	Sí
Opción inexistente	Opción: 9	Solicitar nuevamente la opción.	Sí
Entrada inesperada	Edad: abc	Mostrar error y volver a solicitar.	Sí

Las capturas reales de estas pruebas deben incorporarse como evidencia en el informe. No se deben fabricar capturas: deben tomarse durante la ejecución del programa.

9. Comparación entre código original y mejorado

Aspecto	Código original	Código mejorado
Validación de datos	Mínima o inexistente	Valida tipo, rango y longitud
Manejo de errores	No controla ValueError	Usa try/except
Control de datos	Procesa directamente lo ingresado	Procesa después de validar
Mensajes	No informa errores de conversión	Explica qué debe corregirse
Seguridad y robustez	Puede terminar inesperadamente	Reduce entradas inválidas y fallos de ejecución

La segunda versión es más segura porque no confía directamente en los datos introducidos por el usuario. Primero comprueba que sean del tipo y rango esperado y, si existe un error de conversión, lo controla sin interrumpir todo el programa. Este enfoque coincide con las recomendaciones de validación de entrada y manejo de excepciones de OWASP y de la documentación de Python.

□cite□turn0search0□turn0search1□

10. Evidencias que debe capturar el equipo

- Captura 1: código original abierto en el editor.
- Captura 2: ejecución del código original mostrando una entrada problemática.
- Captura 3: vulnerabilidad identificada en el código o explicación de la validación faltante.
- Captura 4: código mejorado completo o sus funciones principales.
- Captura 5: ejecución del código mejorado rechazando una entrada inválida y solicitándola nuevamente.
- Captura 6: repositorio de GitHub mostrando los archivos y el historial de commits.

11. GitHub y commits

La guía solicita como mínimo cuatro commits: código original, validaciones, manejo de errores y versión final. La secuencia siguiente permite cumplir ese requisito de manera ordenada.

```
git init
git add codigo_original.py
git commit -m "Commit 1: código original"
git add codigo_mejorado.py
git commit -m "Commit 2: incorporación de validaciones"
git add codigo_mejorado.py
```

```
git commit -m "Commit 3: incorporación de manejo de errores"
git add README.md
git commit -m "Commit 4: versión final mejorada"
git branch -M main
git remote add origin https://github.com/TU-USUARIO/TU-REPOSITORIO.git
git push -u origin main
```

Nota: si el docente quiere ver claramente cada etapa, conviene hacer los commits en el orden indicado y no subir todo de una sola vez. Después de crear el repositorio vacío en GitHub, se reemplaza la URL de ejemplo por la URL real del repositorio.

12. Conclusiones

1. La vulnerabilidad más importante fue la ausencia de manejo de errores en las conversiones de datos, porque una entrada no numérica puede detener el programa.
2. Los valores fuera de rango también representan un riesgo porque permiten que el sistema trabaje con información que no cumple las reglas del problema.
3. La estrategia preventiva más efectiva fue combinar validación de tipo, validación de rango y manejo de excepciones.
4. La validación de entradas es importante porque permite que el programa procese solamente información que cumple las condiciones definidas.
5. La actividad demuestra que programación y seguridad están relacionadas: un programa que funciona en condiciones normales no necesariamente está preparado para entradas inesperadas.